

Jayce Kernel

A Constitutional Security Architecture for Deterministic Compute Environments

Technical Whitepaper v0.5 — Multi-Architecture Engine Completion

Jayce Automata Research

August 2026

Abstract.— We present Jayce, a bare-metal deterministic compute kernel and its companion security daemon (Mini Guardian). The kernel is a multi-architecture `no_std` implementation targeting `x86_64` (primary, fully implemented), `aarch64` (display, GIC, input, power, timer), `riscv64`, `mips`, and `arc` (feature-gated), selected at build time through Cargo platform features. Execution is deterministic: a fixed 1,000 Hz tick rate, bounded static queues, and compile-time resource budgets for 44 registered engines. Security is enforced through over a dozen coordinated defense layers, including immutable spine invariants, cryptographic boot trust, an 18-right capability system with constant-time verification, two independent illusion layers (an Outer Bubble protecting the OS from applications and an Inner Shroud protecting the kernel from the OS), a 3-phase device gate (Doorman), a 7-engine safety analysis chain (ISD, PEH, SHCM, CSL, CPC, RCF, TPL), a predictive execution horizon, and a watchdog with circuit-breaker escalation to safe mode. A frozen Shim ABI (v3) exposes only five operations to personalities and userland runtimes and is denied for military/weapons use classes. The kernel emits `JAYCE_TLM` telemetry every 500 ms carrying a SHA-256 engine-lineage fingerprint, which the daemon cross-checks against its own canonical kernel-port build. This paper details the architecture, threat model, defense mechanisms, and verification results.

1. Introduction

1.1 Motivation

Modern compute environments face escalating threats from adversarial machine learning attacks, memory corruption exploits, supply chain compromises, and sophisticated injection techniques targeting AI/ML pipelines. Traditional security approaches—address space layout randomization, stack canaries, sandboxing—operate as independent mitigations without coordinated response. Jayce takes a fundamentally different approach: a constitutional security architecture where every engine operates under explicit, auditable invariants, and violations trigger coordinated escalation through a deterministic recovery pipeline.

1.2 Design Philosophy

Jayce adheres to five non-negotiable principles:

1. **Voluntary and visible:** All security measures are transparent to the operator. No hidden telemetry, no covert channels.
2. **User-mode only:** No OS hooks, no kernel module injection, no syscall interception beyond what the platform provides.
3. **Fail-closed:** When in doubt, deny. When uncertain, quarantine. When corrupted, freeze.
4. **Local-only:** No network communication. No phone-home. No external telemetry. All audit logs are local.
5. **Deterministic:** Fixed tick rate, bounded queues, compile-time resource budgets. No dynamic allocation at runtime.

1.3 Contributions

- A formal architecture for a constitutional security kernel with 44 registered engines across core, system-integrity, hardware, signal-pipeline, and subsystem categories, implemented in a single multi-architecture bare-metal target
- A two-layer illusion isolation system (Outer Bubble + Inner Shroud) providing bidirectional security boundaries
- A frozen, five-operation Shim ABI (v3) with civilian-use-only enforcement, cross-arch constants, and a 10-rule shim firewall
- A 7-engine safety analysis chain (Impossible State Detector, Predictive Execution Horizon, Self-Healing Causality Mesh, Constitutional Shadow Layer, Composable Physical Contracts, Robot Command Fabric, Temporal Proof Ledger) with dependency-ordered evaluation
- A capability system with 18 rights, static bounded table, delegation with depth and TTL bounds, permanent revocation tombstones, and constant-time verification
- An engine-lineage telemetry protocol (JAYCE_TLM) that lets the daemon judge verify the kernel's engine code hash at runtime
- Empirical verification through 74 kernel unit tests plus 23 daemon-port tests (97 kernel-related), 32 trust_core tests, and 899 Mini Guardian tests, all passing with zero warnings

2. Threat Model

2.1 Adversary Capabilities

We consider adversaries with the following capabilities:

- **Software-only adversaries:** Can exploit bugs in application code, craft malicious prompts, inject poisoned training data, and perform timing side-channel attacks within the software boundary.
- **Peripheral adversaries:** Can connect unauthorized USB/HID devices, probe JTAG/SWD debug ports, and attempt DMA attacks via Thunderbolt/PCIe.
- **Network adversaries:** Can attempt WiFi/Bluetooth/RF injection, DNS manipulation, and network-based denial of service.
- **Supply chain adversaries:** Can compromise update mechanisms, tamper with boot images, and inject malicious engine code.
- **AI/ML adversaries:** Can perform prompt injection, context window manipulation, retrieval poisoning, summary injection, and cross-session persistence attacks against AI pipelines.

2.2 Out of Scope

- Physical side-channel attacks requiring proximity to hardware (cold boot DRAM forensics is mitigated but not eliminated)
- Nation-state actors with physical access to the hardware before boot
- Insider attacks by operators with physical access and full knowledge of the security architecture
- Attacks against the Rust compiler toolchain itself

2.3 Security Properties

The architecture enforces the following formal properties:

- **P1 (Invariant Preservation):** The Immutable Spine guarantees that core kernel invariants are never violated. Any violation triggers immediate burn pit disposal.
- **P2 (Capability Enforcement):** All resource access requires a valid capability token. Tokens are verified in constant time and cannot be forged, escalated, or transferred.
- **P3 (Bidirectional Isolation):** The Illusion Bubble prevents applications from accessing host resources; the Inner Shroud prevents the host from manipulating kernel internals.
- **P4 (Deterministic Recovery):** All recovery actions are bounded, deterministic, and auditable. No recovery path requires external communication.
- **P5 (Seal Integrity):** Post-seal, no engine code can be modified without cryptographic verification and rollback capability.

3. Architecture Overview

3.1 System Components

3.1.1 The Kernel

A bare-metal no_std Rust kernel with a multi-architecture HAL.

Property	Value
Architectures	x86_64 (full); aarch64 (display/GIC/input/power/timer); riscv64; mips; arc (feature-gated)
Platform features	platform-x86-pc (default), platform-sbc, platform-mcu, platform-arm64, platform-tablet, platform-phone, platform-watch, platform-riscv64, platform-mips, platform-arc, platform-legacy-fleet
Heap	No runtime heap; static bump allocator for crypto (ed25519-dalek/p256/rand)
Process model	Fixed task table (MAX_TASKS = 16); spawn-only, no fork/exec
Tick rate	1,000 Hz (fixed)
Engine count	44 registered (ENGINE_COUNT) + safety chain
Capability tokens	18-right u32 bitmask (CAP_ALL = 0x0003_FFFF)
Verification	Constant-time (full-table scan)
Memory model	Single-core; the smp feature is a compile-time error until ChauffeurState is migrated
Telemetry	JAYCE_TLM frames every 500 ms with SHA-256 engine lineage

Table 1: Kernel properties

3.1.2 The Sidecar

A userspace security daemon (Mini Guardian) that hosts the canonical kernel code as an in-process port (`jayce_kernel`) and, optionally, launches the real bare-metal kernel inside a lightweight QEMU coprocessor. The daemon advances the kernel tick counter at the 1 kHz cadence, runs the kernel’s host engines under the kernel lock, emits JAYCE_TLM telemetry to `/tmp/jayce-telem.out`, and enforces a judge/enforcer agreement: each tick, a proof-checked Apodeixis mission confirms the coprocessor is alive and that its engine-lineage hash matches the daemon’s port build. The daemon also provides host-side protection (Landlock sandbox, eBPF/XDP filtering, chained seccomp-BPF, root-process ancestry and baseline trust, shadow-backup restoration, antifreeze sentinel) and feeds host intelligence into kernel engines via bridge commands.

3.2 Communication Interfaces

The kernel exposes three deterministic interfaces:

- **Trusted Operator Terminal (COM1 serial):** a bounded, printable-ASCII text shell. Commands: `help/h`, `hw/tier`, `profile`, `status`, `shim`, `n/next`, `p/prev`, `1/2/3`, `r/reset`, `h2/chelp`. It also accepts intent-level `rcf <id> <budget> <op> [args..]` programs submitted to the Robot Command Fabric; every submission is acknowledged as `JAYCE_RCF: id=N ok=B [err=parse]`.
- **DevShell protocol:** a userspace task exchanging fixed `MSG_DEV_*` messages (region alloc/release/list, Doorman stats, capability stats, upgrade request, stream/net stats, and a battery of self-tests: shim conformance, fault containment, domain isolation, determinism stress, microkernel promotion gate, service isolation, IPC fault matrix, flow smoothness, compat stress, illusion penetration, privilege escalation, net abuse, IRQ mediation, PCI/IOMMU/telemetry hardening, and more).

- **Shim ABI v3:** the frozen foreign ABI for personalities and userland runtimes (see Section 5.6).

3.3 Licensing Model

Component	License
Kernel	Jayce Automata Research Source License v1.0 (proprietary)
Sidecar	Jayce Automata Research Source License v1.0 (proprietary)

Table 2: Licensing

The user owns both components. The license is source-available for viewing/study and personal evaluation only; it permits no forking, no modifications, no derivatives, no redistribution, and no commercial use without written agreement. The no-weaponization clause is non-waivable.

4. Security Engines

4.1 Engine Registry

The kernel registers 44 engines in a compile-time budget table. Each budget constrains the engine’s per-tick time (in % of tick budget), memory, and message allowance.

ID	Engine	Time	Mem (B)	Msgs
1	heartbeat	4	0	8
2	watchdog	6	0	8
3	scheduler	12	0	32
4	timer_subsystem	8	0	16
5	input	24	4 096	64
6	echo	8	0	16
7	output_buffer	20	8 192	96
8	latency_engine	32	0	8
9	burst_stabilizer	16	2 048	64
10	burst_governor	12	1 024	32
11	constraint_enforcer	8	0	16
12	automata_monitor	16	2 048	32
13	microburst_buffer	12	2 048	64
14	doorman	24	4 096	64

Table 3: Engine budget table, part 1 of 3

ID	Engine	Time	Mem (B)	Msgs
15	synchronicity	10	0	16
16	traffic_controller	12	0	16
17	burn_pit	8	0	8
18	spine	12	0	16
19	deml	16	2 048	32
20	truth	16	0	16
21	hardware_info	20	4 096	32
22	engine_status	6	0	16
23	boot_trust	10	0	8
24	region_manager	16	2 048	32
25	vitals	8	0	16
26	network	20	4 096	32
27	upgrade	16	8 192	16
28	streams	16	4 096	64

Table 4: Engine budget table, part 2 of 3

ID	Engine	Time	Mem (B)	Msgs
29	usb	24	8 192	64
30	recovery	16	0	16
31	interrupts_subsystem	12	0	16
32	self_model	8	0	8
33	service_supervisor	16	2 048	32
34	illusion_bubble	12	0	16
35	symbol_engine	16	4 096	32
36	pci_config	12	0	16
37	iommu_policy	12	0	8
38	arc_engine_host	16	0	16
39	gamedev	16	8 192	32
40	lightrender	16	4 096	32
41	virtual_device_personalities	16	4 096	32
42	void_pit	8	0	8
43	alien_bridge	12	2 048	32
44	ai_adapter	12	2 048	32

Table 5: Engine budget table, part 3 of 3

4.2 Engine Taxonomy

Engines are organized into categories:

- **Core (9):** heartbeat, watchdog, scheduler, timer_subsystem, latency_engine, burst_stabilizer, burst_governor, constraint_enforcer, output_buffer.

- **System Integrity / Automata:** spine, impossible_state_detector, predictive_execution_horizon, self_healing_causality_mesh, constitutional_shadow_layer, composable_physical_contracts, robot_command_fabric, temporal_proof_ledger, peace_audit, automata_monitor, synchronicity, microburst_buffer, traffic_controller.
- **Hardware / Truth:** hardware_info, truth, echo, self_model, deml, presence, pattern_memory, telemetry, engine_status.
- **Signal Pipeline (event-driven, no module tick):** raw_signal_engine, pattern_detection_engine, signal_typing_engine, signal_semantic_layer, signal_action_layer.
- **Subsystems / Infra:** boot_trust, region_manager, doorman, interrupts_subsystem, service_supervisor, upgrade, streams, network, usb, recovery, pci_config, iommu_policy, virtual_device_personalities.
- **Compatibility adapters (self-gate on late_engines_ready, seq >= 3000):** posix_lite_adapter, filesystem_syscall_adapter, cluster_topology_adapter, hpc_workload_adapter, xr_frame_pacing_adapter, device_breadth_adapter, surface_compression_adapter, process_model_parity_no_fork, virtualization_multitenant_controls_adapter, certification_workflow_readiness_adapter.
- **Personality / I/O adapters:** arc_engine_host, lightrender, gamedev, illusion_bubble, symbol_engine, alien_bridge, ai_adapter, void_pit, piggyback drain.

4.3 Engine Lifecycle

All engines follow a deterministic lifecycle:

Boot -> Registration -> First Tick -> Steady-State Ticks -> [Upgrade | Recovery | Shutdown]

Registration occurs via `engine_status::note_engine_tick()`, which records the engine's first heartbeat. Liveness is determined by `engine_status::is_engine_alive()`, checking whether the engine's last heartbeat is within `ENGINE_LIVENESS_WINDOW_TICKS` (128 ticks = 128 ms at 1 kHz). Engines can be marked unhealthy via `mark_engine_unhealthy()`, which excludes them from liveness checks and triggers recovery.

4.4 Tick Scheduling

Tick evaluation is cadence-scaled per engine and per vital mode. `tick_scaled_tracked!(id, full, constrained, minimal, call)` decides whether an engine runs on a given tick by dividing the engine sequence counter, so in Constrained or Minimal vital mode non-critical engines run at 1-in-2 or 1-in-4 cadence. A `service_mode` flag suppresses echo, output_buffer, network, streams, lightrender, and virtual_device_personalities for headless service deployments. A green-idle lane keeps only safety/determinism-critical engines running during sustained inactivity.

The safety analysis chain runs in dependency order — ISD, PEH, SHCM, CSL, CPC, RCF, TPL — and, in Full mode, executes only when the safety dirty flag is set (set by any safety-relevant event), optimizing tick performance. In deterministic-core and green-idle modes the chain always runs.

5. Defense Layers

5.1 Immutable Spine

The Spine maintains core kernel invariants through per-tick verification. Invariants are computed as a rolling hash over kernel state. Any deviation triggers: an invariant failure counter increment, a burn pit disposal event, a watchdog trip, and a recovery event submission. The Spine operates at tick ID 18, ensuring it runs after all state-modifying engines have completed their work. Spine constants are compile-time: `TIMER_FREQ_HZ = 1_000`, `RENDER_INTERVAL_TICKS = 1_000` (minimum 1 render per second), `PAGE_ROTATE_INTERVAL_TICKS = 5_000`, `UI_ENABLED_DEFAULT = true`, `DETERMINISTIC_CORE_ENABLED_DEFAULT = false`.

5.2 Capability System

Jayce implements a capability-based security model with 18 rights expressed as a u32 bitmask (`CAP_ALL = 0x0003_FFFF`):

Bit	Capability
0	<code>CAP_IPC_SEND</code>
1	<code>CAP_IPC_RECV</code>
2	<code>CAP_MEMORY_ALLOC</code>
3	<code>CAP_MEMORY_FREE</code>
4	<code>CAP_HARDWARE_BIND</code>
5	<code>CAP_INPUT_READ</code>
6	<code>CAP_OUTPUT_WRITE</code>
7	<code>CAP_TELEMETRY_READ</code>
8	<code>CAP_ENGINE_CONTROL</code>
9	<code>CAP_DOORMAN_QUERY</code>
10	<code>CAP_PORT_SCAN</code>
11	<code>CAP_NETWORK_TX</code>
12	<code>CAP_NETWORK_RX</code>
13	<code>CAP_DEVELOPER_CORE</code>
14	<code>CAP_UI_ENABLED</code>
15	<code>CAP_DETERMINISTIC_CORE</code>
16	<code>CAP_STORAGE</code>
17	<code>CAP_BUS</code>

Table 6: Capability token rights (18-right u32 bitmask)

Capability semantics:

- **Static bounded table:** `MAX_CAPABILITY_TOKENS = 64` slots in a static `KernelCell` table.
- **Monotonic token IDs:** tokens are allocated with monotonically increasing IDs; `CAP_ALL` is never minted to a holder.
- **Constant-time verification:** `verify_capability` scans the full table without early exit, preventing timing side channels.

- **Rate-limited verification:** `verify_capability_rate_limited` caps verifications per tick per holder (512/tick/holder), preventing verification-driven DoS.
- **Delegation:** `delegate_capability` supports chains up to `MAX_DELEGATION_DEPTH = 3` with a time-to-live of `DELEGATION_TTL_TICKS = 16_384` ticks; every delegation is a strict subset (attenuation) of the parent rights; revocation of a parent cascades to all descendants.
- **Permanent revocation:** `revoke_capability` writes a tombstone that is never removed; revoked tokens can never be re-issued with the same ID.
- **Privileged issue mask:** a developer-only mask gates issue of `CAP_ENGINE_CONTROL/CAP_MEMORY_ALLOC/CAP_MEMORY_FREE/CAP_BUS`-class rights.
- **Burn-pit integration:** failed verification burns the token.

5.3 IPC Model

IPC is fixed and deterministic:

- **Message:** { kind: u32, arg0: u64, arg1: u64 } — no pointers, no inline data beyond fixed fields.
- **Inbox:** every task has a fixed 8-slot ring buffer (Inbox); push/pop are bounded and fail closed at capacity.
- **Tasks:** `MAX_TASKS = 16` in a static task table; tasks are spawn-only (no fork/exec).
- **Priority:** Low, Normal, High, Critical; the scheduler consumes inboxes by priority order.
- **Events:** a global event queue (`EVENT_QUEUE`) carries `TimerTick`, `KeyboardInput`, `GpioButton`, and `EngineMessage` events.
- **Chauffeur:** the message router enforces per-sender rate limits (32 msgs/tick/sender) and capability checks on every send.

5.4 Memory Model and Region Manager

- **Static state only:** kernel state lives in `KernelCell` (a `MaybeUninit`-backed static cell) and `AtomicU*`; no dynamic structures.
- **Bump allocator:** a static `BumpAllocator` (global allocator, no free) backs `alloc`-using crypto (ed25519-dalek, p256, rand) in a fixed region; runtime kernel engines do not allocate.
- **Region manager:** `MAX_REGIONS = 32` with `TOTAL_BYTES = 1 MiB` of addressable region space; allocation is linear, checked, and fail-closed.
- **Bounded recursion:** all loops have explicit iteration bounds (e.g., recovery processes 16 events/tick).
- **Compiler barriers:** `compiler_fence(SeqCst)` prevents write reordering for security-critical operations.

5.5 Boot Trust

Boot Trust establishes a trust chain from cold boot through the current execution state.

5.5.1 Boot Measurement

At initialization, `boot_trust::init()` computes a 32-bit FNV-1a hash mixing the Shim ABI version, owner fingerprint, Spine measurement fingerprint, page rotation interval, engine count, and timer frequency. This measurement is stored in `BOOT_MEASUREMENT`.

5.5.2 Trust Tiers

Tier	Meaning
Unknown	Boot not yet complete
Observed	Boot complete, not yet verified clean
Trusted	Clean streak exceeded threshold with IOMMU enabled
Revoked	Security violation detected

Table 7: Trust tiers

Trust progression requires `CLEAN_TICKS_FOR_TRUST` (64 clean ticks) plus IOMMU enabled. Any violation immediately drops to Revoked.

5.5.3 Survival Mode

Survival mode is a fail-closed state triggered by port I/O denials (≥ 8), Doorman denials (≥ 4), core mutation denials (≥ 2), service quarantine (≥ 1), IOMMU fault, host integrity tamper, or policy manifest invalidity. In survival mode: core mutations are blocked, network and device binding are denied, network budget is clamped to 4, and strict security mode is enforced. Survival mode implies strict security mode automatically.

5.5.4 Attestation

`build_attestation_record()` produces a signable snapshot containing the boot image hash, configuration hash, IOMMU policy hash, capability schema version, and a development signature. The `tpm_sign()` function is a stub awaiting TPM2 hardware integration; the `dev-signer` feature provides a deterministic signing path without a TPM.

5.6 Frozen Shim ABI (v3)

The Shim is the frozen, deterministic foreign ABI for personalities and userland runtimes. It exposes only five operations and never hands out direct access to kernel internals.

Constant	Value
<code>JAYCE_SHIM_ABI_VERSION</code>	3
<code>JAYCE_SHIM_ABI_MAJOR</code>	1
<code>JAYCE_SHIM_ABI_MINOR</code>	0
Freeze tag	0x5348_5631 (“SHV1”)

Table 8: Shim ABI version constants

5.6.1 The Five Operations

- `send(self_id, token_id, to, msg)` — capability-gated IPC send.
- `recv(self_id, token_id)` — non-blocking inbox pop.
- `map_region(self_id, token_id, size)` — region allocation returning a `JayceShimRegion`.
- `yield_now(self_id)` — cooperative yield.
- `log(self_id, msg)` — deterministic audit log write.

Extended query operations: `attest`, `declare_use_class`, `require_policy`, `query_shape`, `query_compat`, `query_topology`, `require_core_boundary`, `query_conformance`. Message kinds are fixed constants (`SHIM_MSG_NETWORK_TX`, `SHIM_MSG_NETWORK_RX_POLL`, `SHIM_MSG_NETWORK_RX`,

SHIM_MSG_NETWORK_LINK, SHIM_MSG_ATTESTATION, SHIM_MSG_ALIEN_BRIDGE_STATUS,
SHIM_MSG_ALIEN_BRIDGE_RECENT, SHIM_MSG_AI_ADAPTER_ADVISE).

5.6.2 Civilian-Use Enforcement

SHIM_FEATURE_CIVILIAN_USE_ENFORCED (bit 15) marks the ABI as civilian-only: SHIM_USE_CLASS_MILITARY_DENIED = 0xFF_DEAD causes immediate denial at the ABI boundary. declare_use_class rejects any non-civilian class.

5.6.3 Shim Firewall

The Shim flow implements a three-layer architecture (OS-facing Outer illusion, kernel-facing Inner shroud, and the Shim boundary between them) and a 6-step request pipeline. Every request is evaluated against **10 non-negotiable firewall rules** (ShimFirewallRules) before dispatch; if any rule is false the system is treated as misconfigured and the request is denied. A conformance self-test (verify()) asserts all ten rules pass.

5.7 Outer Illusion Bubble

The Outer Illusion Bubble protects the host OS from untrusted applications. It operates as a command gate in the sidecar, intercepting every command before it reaches the kernel.

5.7.1 Command Denylist

The sidecar enforces a 29-entry denylist matching the kernel's IllusionBubble::validate_command(): rm -rf, mkfs, dd, wget, curl, nc, netcat, nmap, ssh, scp, rsync, chmod 777, chown, mount, umount, insmod, modprobe, rmmod, kill -9, shutdown, reboot, halt, poweroff, init, systemctl, service, iptables, ip6tables, ebtables.

5.7.2 Host Monitoring

The sidecar continuously monitors network connections, running processes, and file system changes. Host intelligence is fed into the kernel via the kernel_bridge module.

5.8 Inner Illusion Shroud

The Inner Shroud protects the kernel from the host OS. It is **enabled by default** (SHROUD_ACTIVE: AtomicBool = AtomicBool::new(true)) and operates across 7 security domains:

1. DDoS Detection: Pattern-based DDoS classification with void pit isolation
2. Firewall: SQL injection, XSS, path traversal signature matching
3. USB/HID Protection: Unauthorized device detection and quarantine
4. WiFi/Bluetooth/RF Shielding: Wireless attack surface reduction
5. Prompt Injection Guard: 24 attack families, 200+ patterns
6. Integrity Verification: File and memory integrity checking
7. Capability Enforcement: Runtime capability boundary verification

5.9 Doorman (3-Phase Device Gate)

The Doorman implements a 3-phase device authorization protocol:

1. **Phase 1 (Identity Check):** validates vendor ID, device ID, and serial number against known-bad patterns (Identity* reasons).

2. **Phase 2 (Behavior Check):** analyzes interrupt burst patterns, data flow anomalies, and timing signatures (Behavior* reasons).
3. **Phase 3 (Provenance Check):** verifies device origin, firmware version, and capability requirements (Provenance* reasons).

The deny-reason vocabulary is a typed `DoormanReason` enum spanning identity, behavior, provenance, and binding classes (48+ distinct reasons). Devices that fail any phase are quarantined. The Doorman supports runtime allowlists and denylists ingestible via policy blobs, `authorize_binding()` for capability-masked device access, and lifecycle event recording.

5.10 Constitutional Shadow Layer

The Constitutional Shadow Layer monitors 7 invariant classes every tick: state machine validity, capability consistency, memory region integrity, timing discipline, IPC message validity, transition legality, and resource accounting. The CSL runs in the safety analysis chain and only executes when the safety dirty flag is set, optimizing tick performance.

5.11 Predictive Execution Horizon

The Predictive Execution Horizon projects system hazards 8-20 ticks into the future by analyzing current engine trajectories and resource consumption patterns. It identifies impending resource exhaustion, potential timing violations, capacity pressure buildup, and escalation trajectory toward circuit breaker. The PEH feeds predictions into the watchdog for preemptive mitigation.

5.12 Watchdog + Circuit Breaker

The Watchdog tracks escalation levels and implements the circuit breaker pattern.

5.12.1 Escalation Levels

Level	Meaning	System Response
0	Nominal	Normal operation
1	Warning	Single trip or low-severity event
2	Critical	Repeated trips — most engines still operational
3	Circuit Breaker	Safe mode: minimal vital mode + strict security

Table 9: Watchdog escalation levels

5.12.2 Circuit Breaker Behavior

When escalation reaches level 3, the circuit breaker fires: forces `VitalMode::Minimal` (only heartbeat, watchdog, scheduler, timer, constraint enforcer, doorman, burn pit, spine, engine status, and recovery engines run), enforces strict security mode (fail-closed on bindings, port I/O, mutations), and records the cause with activation counter increment. The circuit breaker is one-shot per escalation cycle, resetting when escalation decays to 0.

5.12.3 Rate-Limited Trips

`trip_rate_limited()` suppresses repeated trips for the same cause within a configurable tick interval, preventing spam-driven escalation.

5.13 Recovery Engine

The Recovery Engine is the event-driven recovery dispatch system. It consumes recovery events from a bounded stream (capacity 64) and executes appropriate remediation:

Event	Code	Remediation	Action
Engine Panic	E01	Mark unhealthy + watchdog escalation	mark_engine_unhealthy
Engine Stall	E02	Mark unhealthy + watchdog escalation	mark_engine_unhealthy
Region Corrupt	E03	Quarantine to QSV + watchdog	quarantine_storage_vault
Device Quarantined	E04	Record isolation + watchdog	isolation counter
Constitutional Violation	E05	Force minimal mode + strict security + freeze	set_vital_mode
Temporal Proof Divergence	E06	Quarantine evidence + freeze lane	quarantine + freeze
Contract Violation	E07	Burn pit disposal + watchdog	burn_pit::dispose
Robot Command Violation	E08	Burn pit disposal + watchdog	burn_pit::dispose

Table 10: Recovery event handling

The recovery engine processes up to 16 events per tick, preventing event queue flooding.

5.14 Upgrade Engine with Rollback

The Upgrade Engine implements a 6-phase hot-swap state machine:

Idle -> Requested -> Draining -> Quiesced -> Upgrading -> Resuming -> Idle

5.14.1 Pre-Upgrade Verification

Before an upgrade: a manifest entry must be registered with a fingerprint commitment, the request must provide a matching fingerprint, and mismatches are recorded as hostile probes and denied.

5.14.2 Rollback Mechanism

On `request_upgrade()`, the old entry point is captured. If the new engine fails its startup heartbeat during the Resuming phase, `rollback_upgrade()` restores the old entry by re-registering in both `engine_status` and the scheduler. `upgrades_rollback()` tracks rollback count.

5.15 Cold Boot Defense

The Cold Boot Defense Engine mitigates DRAM remanence attacks:

- **Two-pass overwrite:** Each region is overwritten with 0xAA, then 0x55, then zeroed
- **Compiler barriers:** `compiler_fence(SeqCst)` prevents optimization of writes
- **Volatile writes:** `write_volatile()` ensures writes reach memory

Lifecycle: register regions via `register_scrub_region()`, trigger via `signal_shutdown()`, scrub completes on next tick.

5.16 Stack Canaries

The Stack Canary Engine provides runtime stack integrity checking. At boot, `stack_canary::init()` generates a 64-bit canary value by mixing the boot measurement, engine count, shim ABI version, and heartbeat tick using FNV-1a. Critical functions place `canary_value()` at the start of their stack frame and call `verify_canary(expected)` before returning. On mismatch, the violation counter increments, the watchdog trips (cause 0x0A), and optionally `verify_canary_or_halt()` forces minimal mode + strict security.

5.17 Debug Port Detection

The Debug Port Detection Engine identifies active hardware debugging tools. On x86_64, it reads Debug Register 7 (DR7) to detect active hardware breakpoints (bits 0-3). On detection: `DEBUGGER_DETECTED` flag is set (one-shot), strict security mode is enforced, watchdog trips (cause 0x0B), and the event is recorded for the peace audit trail.

5.18 Burn Pit

The Burn Pit is the constitutional incinerator. Anything violating invariants is disposed here.

Disposal Kind	Counter
Malformed IPC	IPC counter
Region Violation	REGION counter
Timing Violation	TIMING counter
Invalid Transition	IPC counter
Malformed USB	USB counter
Stream Overflow	OVERFLOW counter (excluded from watchdog)
Illegal Engine Output	Total counter
Corrupt Automata	Total counter

Table 11: Burn pit disposal categories

Stream overflows are classified as capacity-pressure telemetry, NOT constitutional violations, and are excluded from watchdog escalation to preserve signal quality. Every disposal increments the total and category counters, records the last kind, and is visible to the watchdog and the peace audit trail.

5.19 Void Pit

The Void Pit is the quarantine sink for suspicious-but-unproven traffic (e.g., pattern-classified DDoS candidates). Unlike the burn pit, void-pit disposal is reversible for forensic analysis; it never triggers watchdog escalation by itself.

5.20 Multiverse

The Multiverse tracks execution timelines for deterministic reasoning: `MAX_UNIVERSES` = 16 universes, each with at most `MAX_UNIVERSE_CHILDREN` = 4 children, addressed by a 128-bit

`UniverseId` derived deterministically from a parent ID, divergence seed, and tick. Universes support fork/divergence for “what-if” safety analysis without affecting the primary timeline.

5.21 Determinism, Vital Modes, and Power

- **VitalMode:** Full, Constrained, Minimal. Mode selection scales engine cadence and gates features; Minimal is the circuit-breaker safe mode.
- **Deterministic core mode:** when enabled (default off), only essential deterministic engines tick; the UI is forced off and render/interrupt subsystems are suppressed. A `DETERMINISTIC_CORE_ENABLED_DEFAULT` spine constant (false) is the boot default.
- **Green idle:** an idle-hint lane suppresses non-essential work during sustained inactivity, keeping heartbeat, watchdog, scheduler, timer, constraint enforcement, doorman, burn pit, spine, and the safety chain running.
- **Strict security mode:** doubles divisor/attenuation factors and clamps budgets; forced automatically in survival mode and by the circuit breaker.
- **Loop-phase timing:** per-engine tick-time accounting against the compile-time budget table; jitter is reported via p50/p95/p99 runtime statistics behind a debug gate.

5.22 Signal Pipeline and Bridge Engines

The signal pipeline is event-driven rather than tick-driven: `raw_signal_engine` ingests foreign sensor data (e.g., LASV vision frames via `alien_bridge::record_sample` as normalized millimetre amplitudes), `pattern_detection_engine` classifies patterns, and the semantic/action layers map typed signals to capability-gated actions. The `alien_bridge` (ID 43) is the kernel’s raw-signal ingestion point for foreign sensor data, and `ai_adapter` (ID 44) provides deterministic AI advisory that never bypasses capability checks.

5.23 Service Supervisor

The Service Supervisor (`service_supervisor`) monitors `ServiceCandidate` and `DriverCandidate` health, maintains a service table, and drives service mode (`service_mode_enabled()`), which suppresses display/network/personality engines for headless deployments. Quarantined services count toward survival-mode triggers.

5.24 Telemetry and Engine Lineage

The kernel emits `JAYCE_TLM`: frames to COM1 every 500 ms from a fixed 896-byte stack buffer. Each frame carries scheduler statistics, capability deny counts, engine-denial counts, the peace score, the ISD mask, the PEH hazard mask, RCF statistics, and the Temporal Proof Ledger summary (`tpl_r=`, `tpl_v=`, `tpl_h=`). Crucially, every frame carries `l=` – the SHA-256 **engine-lineage** fingerprint computed by `jayce_kernel_src/build.sh` over the canonical engine sources (mirrored in `jayce_kernel/build.rs` for the daemon port). The daemon’s judge/enforcer agreement mission fails closed if the kernel’s `l=` ever disagrees with the daemon’s build.

6. Memory Poisoning Defense

6.1 Overview

The Memory Poisoning Defense is a 5-layer system designed to protect AI/ML pipelines from adversarial manipulation. Unlike traditional cybersecurity threats, memory poisoning attacks target the integrity of AI context, not the execution environment.

6.2 Attack Surface

Modern AI assistants process untrusted input through multiple stages: retrieval (RAG systems fetch documents), context assembly (retrieved content injected into LLM context window), processing (LLM processes combined context), summarization (results summarized and stored), and persistence (summaries retained across sessions). Each stage is vulnerable to adversarial manipulation.

6.3 Defense Layers

6.3.1 RAG Corpus Guard

Monitors incoming document batches for fingerprint drift (FNV-1a fingerprinting detects corpus deviation from baseline), anomaly scoring (statistical deviation from expected distribution), and deduplication (prevents repeated injection of identical poisoned content).

6.3.2 Context Window Guard

Protects the LLM context window from system prompt displacement (detects when injected content pushes system instructions out), token overflow attacks (monitors for content exceeding context limits), and priority inversion (ensures system instructions maintain priority).

6.3.3 Summary Validator

Scans generated summaries for 25 structural injection patterns, authority escalation attempts, and covert channel encoding (steganographic patterns in summary output).

6.3.4 Retrieval Guard

Monitors retrieval-time operations for 32 adversarial trigger patterns, query manipulation (attempts to bias retrieval toward poisoned content), and score inflation.

6.3.5 Persistence Monitor

Tracks cross-session persistence for source trust (maintains trust scores), authorization gates (ensures persistent content was authorized by user), and drift detection (monitors for gradual poisoning across sessions).

7. Prompt Guard

7.1 Attack Family Taxonomy

The Prompt Guard implements 24 attack families with 200+ patterns organized into 5 groups:

7.1.1 Traditional Injection (Families 1-5)

1. Direct injection: “ignore previous instructions”, “forget everything”
2. Role manipulation: “you are now”, “act as”, “pretend to be”
3. Instruction override: “new instructions”, “system prompt”
4. Context manipulation: “conversation so far”, “summary of”
5. Encoding attacks: base64, rot13, hex encoding

7.1.2 Jailbreak Patterns (Families 6-10)

1. DAN (Do Anything Now): “developer mode”, “jailbreak”
2. Character roleplay: “you are”, “act as”, “pretend”
3. Hypothetical scenarios: “hypothetically”, “what if”
4. Academic framing: “research paper”, “academic study”
5. Creative writing: “write a story”, “fictional character”

7.1.3 Exfiltration (Families 11-15)

1. Data extraction: “repeat everything”, “what did I say”
2. System prompt leakage: “show me your instructions”, “print your prompt”
3. Memory probing: “what do you remember”, “do you recall”
4. Context extraction: “summarize the conversation”, “what was discussed”
5. Token extraction: “repeat the above”, “copy the text”

7.1.4 Manipulation (Families 16-19)

1. Confidence tricks: “trust me”, “I’m authorized”
2. Emotional manipulation: “please”, “I need help”, “urgent”
3. Authority spoofing: “as an admin”, “I’m the developer”
4. Urgency pressure: “immediately”, “critical”, “emergency”

7.1.5 AI Memory Poisoning (Families 20-24)

1. Embedding manipulation: Adversarial perturbation of vector embeddings
2. Summary injection: Injecting instructions within summary text
3. Cross-session persistence: Poisoning that survives session boundaries
4. Retrieval triggers: Patterns activated during RAG retrieval
5. Context overflow stress: Tokens designed to exceed context limits

7.2 Semantic Analysis

The Semantic Analyzer implements a 32-dimensional static embedding model with 32 pre-computed term vectors, class centroids for DataTheft, SelfModifying, PromptInjection, and MemoryPoisoning, 16 additional memory poisoning term vectors (rag, corpus, vector, embedding, perturbation, poison, retrieval, summary, persistent, memory, session, context, drift, sleeper, activate), and cosine similarity classification against centroids.

8. Pre-Seal Threat Audit

8.1 Methodology

A comprehensive threat audit was conducted across 12 categories prior to cryptographic sealing.

N	Category	Findings	Critical	High	Medium
1	Resource Exhaustion	3	0	0	2
2	Race Conditions	2	0	0	1
3	DoS Attacks	3	0	0	2
4	Side-Channel Timing	3	0	0	2
5	Adversarial ML	2	0	0	1
6	Recovery & Rollback	5	0	3	2
7	Upgrade Integrity	4	1	2	1
8	Boot Trust	4	0	2	1
9	Cold Boot	2	0	0	1
10	IPC Security	3	0	1	1
11	Hardware Debug	2	0	1	0
12	Post-Seal Planning	2	0	1	0

Table 12: Pre-seal threat audit findings by category

8.2 Remediation Status

8.2.1 Completed

Gap	Severity	Remediation
Recovery engine stubs	HIGH	8 handler implementations with quarantine + watchdog integration
Watchdog circuit breaker	HIGH	Level 3 safe mode entry, one-shot per cycle, decay reset
Upgrade rollback	HIGH	Save old entry on request, restore on StartupFail
Engine health tracking	MEDIUM	mark_engine_unhealthy / set_engine_healthy API
Cold boot defense	MEDIUM	Two-pass 0xAA/0x55/0x00 scrubbing with compiler barriers
Stack canaries	MEDIUM	Boot-time generation, per-frame verification
Per-IPC rate limiting	MEDIUM	Already implemented in chauffeur (32 msgs/tick/sender)
JTAG detection	MEDIUM	DR7 register probe, strict security on detection
Build fix (SHIM_FEATURE_EMBEDDED_PACK0W		Added missing constant

Table 13: Completed remediations

8.2.2 Resolved in Rev 4 Audit

Remediation	Severity	Resolution
ed25519 signing	HIGH	RESOLVED: Implemented using ed25519-dalek in no_std alloc mode for signature validation
Lockfile integrity	MEDIUM	RESOLVED: Relocated PID lock from /tmp to root-only /var/lib/jayce/miniguard/miniguard.pid with atomic pidfd validation
Quantum-resistant signing	LOW	POST-1.0 ROADMAP: Crypto-agile protocol tags reserved for ML-DSA (Dilithium) / SPHINCS+

Table 14: Completed remediations (Rev 4)

9. Verification Results

9.1 Kernel Build

The kernel builds as a workspace member with zero errors and zero warnings across the supported platform features.

Property	Value
Primary target	x86_64-unknown-none (boot path, full HAL)
Portable HALs	aarch64 (display, GIC, input, power, timer), riscv64, mips, arc
Smoke payloads	riscv64_smoke, arc_smoke, mips_smoke
Errors	0
Warnings	0

Table 15: Build status

9.2 Test Suites

9.2.1 Jayce Kernel (74 tests)

Module	Tests	Status
Spine invariants	13	PASS
Capabilities	9	PASS
Impossible State Detector	14	PASS
Predictive Execution Horizon	11	PASS
Self-Healing Mesh	11	PASS
Streams	14	PASS
Piggyback channels	9	PASS
Truth engine	5	PASS
Network	3	PASS
Doorman	2	PASS
Void Pit	2	PASS
KernelCell	8	PASS
Other	1	PASS
Total	74	PASS (0 failures)

Table 16: Jayce Kernel test results

The daemon-port crate (`jayce_kernel`) additionally runs 23 tests over the synced host-engine sources, including the engine-lineage agreement and temporal proof ledger.

9.2.2 `trust_core` (32 tests)

Test Group	Tests	Status
Memory poisoning defense	27	PASS
Additional trust/guardian coverage	5	PASS
Total	32	PASS (0 failures)

Table 17: `trust_core` test results (shared sidecar library)

Mini Guardian adds 899 tests (unit, attack-simulation scenarios, and ignored live-validation paths).

9.3 Engine Metrics

Metric	Value
Total engines	44 registered + safety chain
Safety analysis chain	7 engines (ISD, PEH, SHCM, CSL, CPC, RCF, TPL)
Recovery event capacity	64 events
Quarantine vault capacity	64 entries
Capability token bits	18 (u32 bitmask)
Capability table size	64 tokens (static)
Delegation depth	3 max; TTL 16 384 ticks
IPC tasks	16 max; 8-slot inboxes
Upgrade slots	8 concurrent
Manifest entries	16 maximum
Watchdog escalation levels	0-3
Circuit breaker threshold	3
Stack canary size	64-bit
Debug detection method	DR7 register probe
Telemetry cadence	JAYCE_TLM every 500 ms
Regions	32 max; 1 MiB space

Table 18: Engine metrics

10. Implementation Notes

10.1 Determinism Guarantees

All resource budgets are compile-time constants in the spine’s engine-budget table. Queue sizes use `const` arrays with fixed capacity. Tick budgets use deterministic division with `is_power_of_two` optimization. Memory uses static `KernelCell` with `MaybeUninit` backing. The kernel’s single-threaded execution model eliminates data races by construction; the `smp` feature flag produces a compile-time error until the `Chauffeur` single-core facade is migrated.

10.2 Memory Model

- **Static allocation only:** All state is static with `KernelCell` or `AtomicU` types.
- **Bump allocator:** A static `BumpAllocator` (no free) provides `alloc` for crypto crates; kernel engines do not allocate at runtime.
- **Bounded recursion:** All loops have explicit iteration bounds (e.g., recovery processes 16 events/tick).
- **Compiler barriers:** `compiler_fence(SeqCst)` prevents write reordering for security-critical operations.

10.3 Cryptographic Primitives

Primitive	Use	Status
FNV-1a 64-bit	Content hashing, fingerprinting	Implemented
FNV-1a 32-bit	Boot measurement, config hashing	Implemented
SHA-256	Temporal Proof Ledger chaining, engine-lineage, H() hashing	Implemented
HMAC-SHA256	Truth engine integrity, gossip mutual auth	Implemented
Ed25519	Upgrade manifest / policy signing + verification	Implemented (ed25519-dalek, no_std alloc)
P-256 ECDSA	Certificate-style signatures	Available (p256 crate)
TPM2	Attestation signing	Stub (awaiting hardware)
ML-DSA / SPHINCS+	Quantum-resistant signing	Research phase

Table 19: Cryptographic primitives status

11. Related Work

- **seL4:** Formally verified microkernel; Jayce differs by targeting AI/ML security rather than general-purpose OS isolation
- **TrustZone/ARMCCA:** Hardware-enforced trusted execution; Jayce provides software-enforced isolation on x86_64, with aarch64 HAL support
- **Intel SGX/TDX:** Hardware enclaves with attestation; Jayce provides attestation without hardware dependency
- **RISC-V Keystone:** Open-source TEE; Jayce targets existing hardware, with riscv64 HAL support
- **Prompt injection defenses:** Prior work focuses on input filtering; Jayce integrates prompt defense into a broader constitutional security architecture

12. Conclusion

We have presented Jayce, a constitutional security architecture comprising a deterministic multi-architecture kernel (44 registered engines across core, system-integrity, hardware, signal-pipeline, and subsystem categories, plus a 7-engine safety analysis chain) and a companion daemon (Mini Guardian) providing host-side protection and judge/enforcer agreement over a shared engine-lineage fingerprint. The architecture implements over a dozen distinct defense layers, from immutable spine invariants through predictive execution horizon to cold boot memory scrubbing, enforced by an 18-right constant-time capability system and a frozen five-operation Shim ABI. The kernel passes 74 unit tests with zero failures; the daemon port adds 23; trust_core passes 32; and Mini Guardian adds 899 tests.

The remaining open items (TPM2 attestation signing, quantum-resistant signing plan, SMP migration) represent incremental hardening that does not affect the core security properties. The architecture is designed for these upgrades: the upgrade engine’s rollback mechanism,

the manifest commitment model, and the capability token system all support cryptographic primitive replacement without architectural changes.

13. Appendix A: Engine Tick Order

The full-mode tick sequence (engine IDs as tracked by `engine_status`; scaled engines run at reduced cadence in Constrained/Minimal modes):

ID	Engine	ID	Engine
1	Heartbeat	7	Output Buffer
2	Watchdog	30	Interrupts Subsystem (anomaly model)
3	Scheduler	8	Latency Engine
4	Timer Subsystem	9	Burst Stabilizer
5	Input	10	Burst Governor
6	Echo	11	Constraint Enforcer
12	Automata Monitor	13	Microburst Buffer
14	Doorman	15	Synchronicity
16	Traffic Controller	17	Burn Pit
42	Void Pit	18	Spine
19	DEML	20	Truth
21	Hardware Info	22	Self Model
23	Engine Status		

Table 20: Front half of the engine tick sequence

After the front half, the safety analysis chain (ISD, PEH, SHCM, CSL, CPC, RCF, TPL) runs when the safety dirty flag is set, followed by `peace_audit`, presence, the compatibility adapters, and telemetry. Then the tail engines:

ID	Engine	ID	Engine
24	Boot Trust	25	Region Manager
26	Vitals	27	Network
28	Upgrade	29	Streams
30	Service Supervisor	31	Recovery
32	ARC Engine Host	40	LightRender
41	Virtual Device Personalities	43	Alien Bridge
44	AI Adapter		

Table 21: Tail of the engine tick sequence (piggyback drain at end)

Safety analysis chain (dependency-ordered): ISD, PEH, SHCM, CSL, CPC, RCF, TPL.

14. Appendix B: Capability Token Rights

Bit	Capability	Description
0	CAP_IPC_SEND	Send IPC messages
1	CAP_IPC_RECV	Receive IPC messages
2	CAP_MEMORY_ALLOC	Allocate kernel memory
3	CAP_MEMORY_FREE	Free kernel memory
4	CAP_HARDWARE_BIND	Bind to hardware
5	CAP_INPUT_READ	Read input devices
6	CAP_OUTPUT_WRITE	Write output devices
7	CAP_TELEMETRY_READ	Read telemetry
8	CAP_ENGINE_CONTROL	Control engines
9	CAP_DOORMAN_QUERY	Query Doorman status
10	CAP_PORT_SCAN	Scan ports
11	CAP_NETWORK_TX	Network transmit
12	CAP_NETWORK_RX	Network receive
13	CAP_DEVELOPER_CORE	Developer core access
14	CAP_UI_ENABLED	UI enabled
15	CAP_DETERMINISTIC_CORE	Deterministic core access
16	CAP_STORAGE	Storage access
17	CAP_BUS	Bus access

Table 22: Capability token rights

15. Appendix C: Watchdog Cause Codes

Code	Source	Description
0x01	Recovery	Engine panic
0x02	Recovery	Engine stall
0x03	Recovery	Region corrupt
0x04	Recovery	Device quarantined
0x05	Recovery	Constitutional violation
0x06	Recovery	Temporal proof divergence
0x07	Recovery	Contract violation
0x08	Recovery	Robot command violation
0x0A	Stack Canary	Canary corruption detected
0x0B	Debug Port	Hardware debugger detected

Table 23: Watchdog cause codes

16. Appendix D: Security Metrics

Metric	Value
Recovery events handled	Atomic counter, per-session
Engines restarted	Atomic counter, per-session
Regions quarantined	Atomic counter, per-session
Devices isolated	Atomic counter, per-session
Freeze actions taken	Atomic counter, per-session
Watchdog trips	Per-cycle counter
Circuit breaker activations	Atomic counter
Stack canary violations	Atomic counter
Debug detection probes	Atomic counter
Upgrade rollbacks	Atomic counter
Manifest denials	Atomic counter
Cold boot scrub attempts	Atomic counter
Capability denials	Atomic counter (exposed in JAYCE_TLM cap_d)
Engine denials	Atomic counter (exposed in JAYCE_TLM ep_d)
Engine-lineage hash	SHA-256 (exposed in JAYCE_TLM l=)

Table 24: Security metrics